

■ PyFlow RPA

Guia de Continuidade para IA

Documento de orientação para uma IA assistente continuar o desenvolvimento do PyFlow RPA a partir do ponto atual.

Este documento foi criado para que, ao mudar de sessão ou de IA, o desenvolvimento continue sem perda de contexto. Leia tudo antes de fazer qualquer modificação no projeto.

1. O que é este projeto?

PyFlow RPA é uma ferramenta desktop de automação visual (estilo UiPath/n8n) feita em Python. O usuário arrasta blocos para um canvas, configura parâmetros e executa fluxos de automação **sem escrever código**.

Componente	Tecnologia / Detalhe
Interface gráfica	PySide6 (Qt)
Automação web	Selenium + ChromeDriver
API local	FastAPI + Uvicorn (porta padrão 8080, auto-detect)
Persistência de fluxos	JSON na pasta flows/
Tema visual	Catppuccin Mocha — dark only (modo claro foi REMOVIDO)
Python	3.11+
Caminho do projeto	C:\Users\Gleudson\pasta4\Python\pyflow\pyflow\

2. Os 5 arquivos que você SEMPRE modifica ao adicionar um novo bloco

Toda vez que um novo bloco é criado, esses arquivos precisam ser tocados:

FILE 1 — `blocks//nome_do_bloco.py` → CRIAR

- Crie o arquivo do bloco aqui. Todo bloco herda de `BaseBlock` (`blocks/base_block.py`).
- Atributos obrigatórios de classe: `name` (str), `description` (str), `category` (str), `params_schema` (list)
- Método obrigatório: `execute(self, params: dict) -> dict`
- Retorno mínimo: `{"success": bool, "message": str}`
- Retorno com dados (só blocos de controle): `{"success": True, "message": "...", "data": {...}}`
- Categorias válidas: "Navegador", "Controle", "Arquivos", "Integração", "Sistema"
- Sempre comece `execute()` chamando `self.validate_params(params)`

```
class MeuBloco(BaseBlock):
    name = "Nome exibido"
    description = "Descricao curta"
    category = "Controle"
    params_schema = [
        {"name": "param1", "label": "Rotulo", "type": "str",
         "required": True, "default": "", "placeholder": "Exemplo"}
    ]

    def execute(self, params: dict) -> dict:
        errors = self.validate_params(params)
        if errors:
            return {"success": False, "message": "\n".join(errors)}
        # sua logica aqui
        return {"success": True, "message": "OK"}
```

FILE 2 — `ui/block_panel.py` → MODIFICAR

- Painel lateral onde o usuário vê e arrasta os blocos.
- Passo 1: adicionar import da nova classe no topo do arquivo.
- Passo 2: adicionar a classe na lista AVAILABLE_BLOCKS na posição correta por categoria.
- A ordem em AVAILABLE_BLOCKS é exatamente a ordem exibida na interface.

```
from blocks.minha_categoria.meu_bloco import MeuBloco

AVAILABLE_BLOCKS = [
    ...
    MeuBloco,    # adicione na posicao correta
    ...
]
```

FILE 3 — ui/canvas.py → MODIFICAR

- Canvas visual onde os blocos ficam após serem arrastados.
- Passo 1: adicionar import da nova classe no topo.
- Passo 2: adicionar "NomeDaClasse": NomeDaClasse no dicionário BLOCK_REGISTRY.
- SE o bloco for de escopo (Loop, Se, etc.): também atualizar _SCOPE_COLORS, _OPEN_TYPES ou _CLOSE_TYPES.

```
BLOCK_REGISTRY = {
    ...
    "MeuBloco": MeuBloco,
    ...
}

# Para blocos de escopo:
_SCOPE_COLORS = {"MeuBloco": "#fab387"} # cor da borda
_OPEN_TYPES = frozenset {..., "MeuBloco"})
_CLOSE_TYPES = frozenset {..., "EndMeuBloco"})
```

FILE 4 — engine/runner.py → MODIFICAR (só blocos de controle)

- Motor de execução principal. Só mexa aqui se o bloco tiver lógica de loop, condição ou escopo.
- _SKIP_TYPES: frozenset com marcadores de fim que o runner ignora no loop principal.
- Se criar EndMeuBloco, adicione em _SKIP_TYPES.
- NUNCA faça _run_sub iterar steps manualmente — ele deve sempre chamar self.run(steps).

```
_SKIP_TYPES = frozenset({
    "EndLoopBlock", "EndForEachBlock",
    "EndIfBlock", "ElseBlock",
    "EndMeuBloco"    # adicione aqui
})

def _run_sub(self, steps, base_index):
    return self.run(steps) # SEMPRE assim – nao mude
```

FILE 5 — flows/teste_nome_do_bloco.json → CRIAR

- Convenção do projeto: toda funcionalidade nova ganha um fluxo de teste JSON.
- Salvar em flows/ com prefixo 'teste_'.
- O fluxo deve exercitar o bloco em pelo menos 2-3 cenários diferentes.

3. Arquitetura de Controle de Fluxo ■ MUITO IMPORTANTE

Os blocos **Loop**, **Para Cada** e **Se NÃO** usam mais o parâmetro **blocks_count** para saber quantos blocos são internos. O Runner detecta automaticamente os marcadores de fim.

Arquitetura de marcadores:

- Loop (Repetir) → abre escopo → usa **EndLoopBlock** para fechar
- Para Cada (For Each) → abre escopo → usa **EndForEachBlock** para fechar
- Condição (Se) → abre escopo → pode ter **ElseBlock** no meio → usa **EndIfBlock** para fechar
- Todos os marcadores de fim estão em **_SKIP_TYPES** no runner e são ignorados pelo loop principal

Como o runner detecta o escopo:

O bloco de abertura retorna **data** com uma chave especial. O runner detecta essa chave, chama **_find_scope_end()** para achar o marcador de fim correspondente e executa os blocos internos via **_run_sub()**. O aninhamento funciona porque **_run_sub** chama **self.run()** recursivamente.

```
# Loop retorna:
{"success": True, "data": {"loop": True, "times": 5}}

# Para Cada retorna:
{"success": True, "data": {"foreach": True, "items": [...], "variable_name": "item"}}

# Se retorna:
{"success": True, "data": {"if_result": True}}
```

4. Onde ficam as variáveis do fluxo

O contexto de variáveis é um **dict estático** em **ExtractTextBlock._context** (arquivo: **blocks/browser/extract_text.py**). É compartilhado entre todos os blocos durante a execução.

```
from blocks.browser.extract_text import ExtractTextBlock

# Ler uma variável:
valor = ExtractTextBlock._context.get("minha_var", "")

# Escrever uma variável:
ExtractTextBlock._context["minha_var"] = "novo_valor"
```

Sintaxe nos parâmetros	O que faz
{{nome_da_variavel}}	Substituído pelo valor da variável antes de executar (via <code>resolve_params()</code>)
{{ASSET:chave}}	Buscado no <code>assets.json</code> via <code>AssetManager</code> — para credenciais

5. Armadilhas — erros comuns para NUNCA cometer

■ ARMADILHA 1: Não use `blocks_count` nos blocos de controle

Os blocos Loop, Para Cada e Se não usam mais `blocks_count`. O Runner detecta os marcadores automaticamente. Nunca reintroduza esse parâmetro — vai quebrar o fluxo.

■ ARMADILHA 2: `_run_sub` deve SEMPRE chamar `self.run()`

Se `_run_sub` iterar os steps manualmente sem chamar `run()`, os aninhamentos de Se/Loop/ForEach vão quebrar — ambos os ramos do If serão executados. Correto: `def _run_sub(self, steps, base_index): return self.run(steps)`

■ ARMADILHA 3: Modo claro foi removido completamente

`theme_manager.py` tem apenas o tema escuro Catppuccin Mocha. Não existe toggle de tema. Não reintroduza `is_dark()`, `toggle()` ou `LIGHT palette` — foram deletados.

■ ARMADILHA 4: A API usa FastAPI, não Flask

`api_server.py` usa fastapi + uvicorn. Flask foi removido do projeto e do `requirements.txt`. Não use `'from flask import ...'` em nenhum lugar.

■ ARMADILHA 5: Não crie outro dict de contexto

Não crie uma variável global paralela para armazenar estado do fluxo. Use sempre `ExtractTextBlock._context`.

6. Estado atual do projeto

Item	Valor
Total de blocos	51
Categoria Navegador	21 blocos
Categoria Controle	15 blocos (Se, Senão, FimSe, Loop, FimLoop, ParaCada, FimParaCada, ...)
Categoria Arquivos	8 blocos (inclui Ler Texto de PDF)
Categoria Integração	3 blocos
Categoria Sistema	5 blocos (inclui Texto para Voz TTS)
API	FastAPI + Uvicorn — porta 8080 (auto-detect se ocupada)
Tema	Catppuccin Mocha (dark only)
ConditionalRetry	Por categoria: timeout, network, stale, notfound, invalid, custom
Fluxos de exemplo	30+ em flows/
Endpoints da API	/flows /run /status /history /stop /dashboard /docs

7. Checklist — adicionando um bloco simples

- [] 1. Criar `blocks//nome_do_bloco.py` herdando de `BaseBlock`
- [] 2. Preencher `name`, `description`, `category`, `params_schema`
- [] 3. Implementar `execute()` retornando `{"success": bool, "message": str}`
- [] 4. Importar e adicionar em `ui/block_panel.py` → `AVAILABLE_BLOCKS`
- [] 5. Importar e adicionar em `ui/canvas.py` → `BLOCK_REGISTRY`
- [] 6. Criar `flows/teste_nome_do_bloco.json`

8. Checklist — adicionando um bloco de escopo (Loop, Se, TryCatch...)

- [] 1. Criar bloco de abertura — retorna data com chave identificadora
- [] 2. Criar bloco de fim (`EndMeuBloco`) — `execute()` retorna só `success/message`
- [] 3. Adicionar `EndMeuBloco` em `runner.py` → `_SKIP_TYPES`
- [] 4. Em `runner.py` → `run()`, detectar `data.get('meu_id')` e usar `_find_scope_end()`
- [] 5. Adicionar `MeuBloco` em `canvas.py` → `_SCOPE_COLORS` e `_OPEN_TYPES`
- [] 6. Adicionar `EndMeuBloco` em `canvas.py` → `_CLOSE_TYPES`
- [] 7. Adicionar ambos em `block_panel.py` → `AVAILABLE_BLOCKS`
- [] 8. Adicionar ambos em `canvas.py` → `BLOCK_REGISTRY`
- [] 9. Criar `flows/teste_meu_bloco.json`

9. Como se comunicar com o dono do projeto

- Sempre confirme o que será feito **antes** de modificar arquivos
- Sempre peça para ver o arquivo antes de editá-lo — o usuário cola o conteúdo no chat
- Toda funcionalidade nova deve ganhar um fluxo de teste JSON em `flows/`
- Se precisar de um arquivo que não foi fornecido, diga exatamente qual e por quê
- O dono do projeto chama-se **Gleidson**
- O projeto é em **português** — nomes de blocos, mensagens e labels em pt-BR
- Prefira perguntar do que assumir — melhor confirmar um detalhe do que refazer tudo